

GRU MODEL

WHEEL FLANGE

```
In [ ]: import pandas as pd
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
```

```
In [ ]: df = pd.read_csv('wheel.csv')
df
```

Out[]:

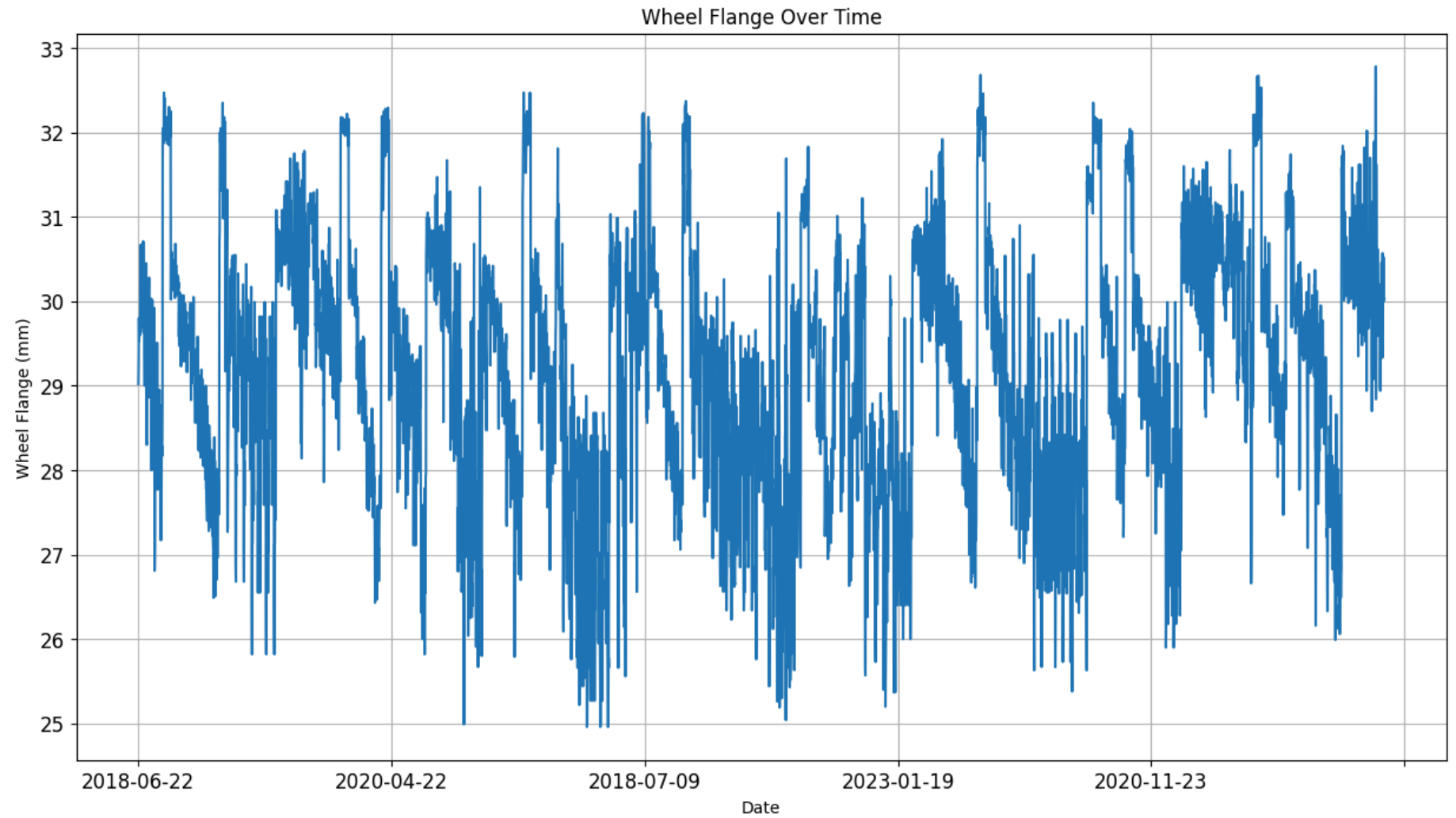
	Date	TrainID	Car	Bogie	Axle	WheelDiameter	WheelFlange	Failure	Quarter	FailuresPerQuarter	DaysInQuarter	MTBF
0	2018-06-22	T01	M1	BOG#1	X1-LHS	843.22	29.02	0	2018Q2	0	91	0.000000
1	2018-06-22	T01	M1	BOG#1	X1-RHS	843.93	29.26	0	2018Q2	0	91	0.000000
2	2018-06-22	T01	M1	BOG#1	X2-LHS	844.14	29.80	0	2018Q2	0	91	0.000000
3	2018-06-22	T01	M1	BOG#1	X2-RHS	844.76	29.53	0	2018Q2	0	91	0.000000
4	2018-06-22	T01	M1	BOG#2	X3-LHS	844.86	29.80	0	2018Q2	0	91	0.000000
...	...	...	...	...	...	...	...	...	...	...	...	...
4918	2025-03-13	T58	M2	BOG#1	X2-RHS	818.00	30.07	0	2025Q1	11	90	147.272727
4919	2025-03-13	T58	M2	BOG#2	X3-LHS	816.30	30.52	0	2025Q1	11	90	147.272727
4920	2025-03-13	T58	M2	BOG#2	X3-RHS	812.10	30.00	0	2025Q1	11	90	147.272727
4921	2025-03-13	T58	M2	BOG#2	X4-LHS	812.10	30.01	0	2025Q1	11	90	147.272727
4922	2025-03-13	T58	M2	BOG#2	X4-RHS	808.10	30.03	0	2025Q1	11	90	147.272727

4923 rows × 12 columns

```

In [ ]: #plot the time series data
df.set_index('Date', inplace=True) # Make sure 'Date' is parsed to datetime
df['WheelFlange'].plot(figsize=(15, 8), title='Wheel Flange Over Time', fontsize=12)
plt.ylabel('Wheel Flange (mm)')
plt.grid(True)

```



```
In [ ]: len(df)
```

```
Out[ ]: 4923
```

TRAIN TEST SPLIT

```
In [ ]: test_size = 3938
        test_ind = len(df) - test_size
```

```
In [ ]: train = df.iloc[:test_ind]
        test = df.iloc[test_ind:]
```

```
In [ ]: #Feature scaling
        from sklearn.preprocessing import MinMaxScaler
        numerical_cols = ['WheelFlange']
        scaler = MinMaxScaler()
        scaler.fit(train[numerical_cols])
```

```
Out[ ]: ▼ MinMaxScaler ⓘ ?
```

```
MinMaxScaler()
```

```
In [ ]: from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator
        length = 12

        # Scale the training data and assign it to scaled_train
        scaled_train = scaler.transform(train[numerical_cols])

        generator = TimeseriesGenerator(scaled_train, scaled_train, length=length, batch_size=1)
```

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, LSTM, GRU
```

```
In [ ]: from keras.layers import Dropout
        from keras.optimizers import Adam

        gru_model = Sequential()

        # Define the number of features
        n_features = 1 # Get the number of columns from the scaled training data

        gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
        gru_model.add(Dropout(0.2))
```

```
gru_model.add(Dense(1))

optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.summary()
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 50)	7,950
dropout (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 8,001 (31.25 KB)

Trainable params: 8,001 (31.25 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: # Scale the test data using the same scaler fitted on the training data
scaled_test = scaler.transform(test[numerical_cols])

# Create a TimeseriesGenerator for the validation data
# The targets for the validation generator should be the scaled test data itself
validation_generator = TimeseriesGenerator(scaled_test, scaled_test, length=length, batch_size=1)

# Now you can call the fit method with the validation_generator
gru_model.fit(generator, epochs=20, validation_data=validation_generator)
```

Epoch 1/20

```
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignored.  
self._warn_if_super_not_called()
```

```

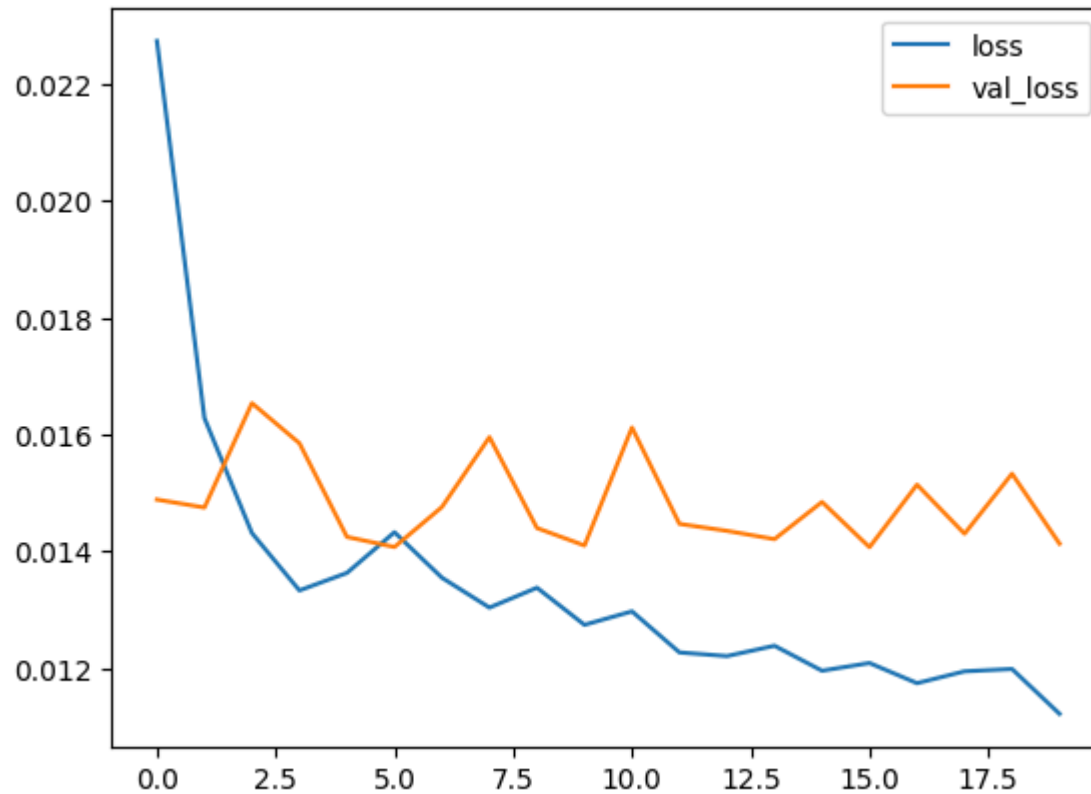
973/973 ————— 22s 20ms/step - loss: 0.0426 - val_loss: 0.0149
Epoch 2/20
973/973 ————— 27s 28ms/step - loss: 0.0154 - val_loss: 0.0148
Epoch 3/20
973/973 ————— 17s 18ms/step - loss: 0.0161 - val_loss: 0.0165
Epoch 4/20
973/973 ————— 19s 19ms/step - loss: 0.0136 - val_loss: 0.0159
Epoch 5/20
973/973 ————— 17s 18ms/step - loss: 0.0145 - val_loss: 0.0143
Epoch 6/20
973/973 ————— 18s 18ms/step - loss: 0.0151 - val_loss: 0.0141
Epoch 7/20
973/973 ————— 28s 29ms/step - loss: 0.0130 - val_loss: 0.0148
Epoch 8/20
973/973 ————— 27s 28ms/step - loss: 0.0133 - val_loss: 0.0160
Epoch 9/20
973/973 ————— 17s 18ms/step - loss: 0.0129 - val_loss: 0.0144
Epoch 10/20
973/973 ————— 18s 18ms/step - loss: 0.0132 - val_loss: 0.0141
Epoch 11/20
973/973 ————— 28s 28ms/step - loss: 0.0117 - val_loss: 0.0161
Epoch 12/20
973/973 ————— 27s 28ms/step - loss: 0.0128 - val_loss: 0.0145
Epoch 13/20
973/973 ————— 17s 18ms/step - loss: 0.0123 - val_loss: 0.0144
Epoch 14/20
973/973 ————— 20s 18ms/step - loss: 0.0113 - val_loss: 0.0142
Epoch 15/20
973/973 ————— 17s 18ms/step - loss: 0.0130 - val_loss: 0.0148
Epoch 16/20
973/973 ————— 18s 18ms/step - loss: 0.0122 - val_loss: 0.0141
Epoch 17/20
973/973 ————— 27s 28ms/step - loss: 0.0121 - val_loss: 0.0151
Epoch 18/20
973/973 ————— 17s 18ms/step - loss: 0.0120 - val_loss: 0.0143
Epoch 19/20
973/973 ————— 29s 29ms/step - loss: 0.0119 - val_loss: 0.0153
Epoch 20/20
973/973 ————— 17s 18ms/step - loss: 0.0111 - val_loss: 0.0141

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f00335d7950>
```

```
In [ ]: losses = pd.DataFrame(gru_model.history.history)
losses.plot()
```

Out[]: <Axes: >



```
In [ ]: test_gru_predictions = []

first_eval_batch = scaled_train[-length:]
current_batch = first_eval_batch.reshape((1, length, n_features))

for i in range(len(test)):
    #get prediction 1 time stamp ahead
    current_pred = gru_model.predict(current_batch, verbose=0)[0]
    #store prediction
    test_gru_predictions.append(current_pred)
```



```
#update batch to now include prediction and drop first value  
current_batch = np.append(current_batch[:,1:,:],[[current_pred]],axis=1)
```

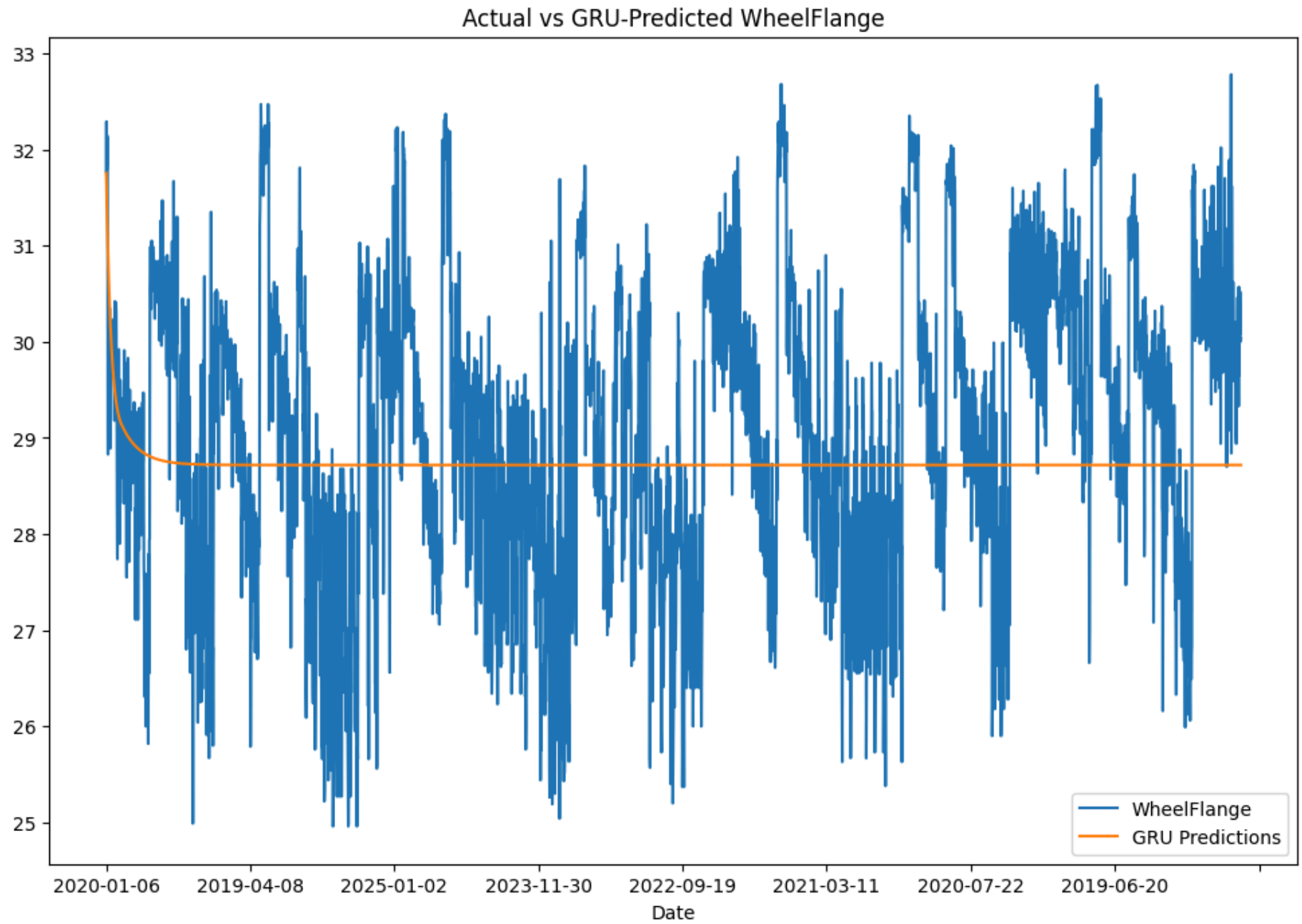
```
In [ ]: true_prediction = scaler.inverse_transform(test_gru_predictions)  
test['GRU Predictions'] = true_prediction  
test[['WheelFlange', 'GRU Predictions']].plot(figsize=(12,8), title='Actual vs GRU-Predicted WheelFlange')
```

```
<ipython-input-14-9422e04173e1>:2: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
test['GRU Predictions'] = true_prediction
```

```
Out[ ]: <Axes: title={'center': 'Actual vs GRU-Predicted WheelFlange'}, xlabel='Date'>
```

**EVALUATION**

```
In [ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from numpy import sqrt

# Inverse transform predictions if needed
true_prediction = scaler.inverse_transform(test_gru_predictions)
test['GRU Predictions'] = true_prediction.flatten()

# Inverse transform actual values if WheelDiameter is still scaled
# Uncomment the line below if your 'WheelDiameter' column is still scaled
# test['WheelDiameter'] = scaler.inverse_transform(test[['WheelDiameter']])

# Ground truth and prediction arrays
y_true = test['WheelDiameter'].values
y_pred = test['GRU Predictions'].values

# Metrics calculation
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mse = mean_squared_error(y_true, y_pred)
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
r2 = r2_score(y_true, y_pred)

# Print results
print(f"MAE: {mae:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MSE: {mse:.4f}")
print(f"MAPE: {mape:.2f}%")
print(f"R²: {r2:.4f}")
```

```
MAE: 781.1536
RMSE: 781.5505
MSE: 610821.1157
MAPE: 96.45%
R²: -984.8790
```

```
<ipython-input-15-acdd81adeaf5>:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
test['GRU Predictions'] = true_prediction.flatten()
```

```
In [ ]: gru_model = Sequential()
gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
gru_model.add(Dropout(0.2))

gru_model.add(Dense(1))

optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.fit(generator, epochs=4)
```

Epoch 1/4

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(**kwargs)
```

```
973/973 ————— 10s 7ms/step - loss: 0.0782
```

Epoch 2/4

```
973/973 ————— 8s 8ms/step - loss: 0.0177
```

Epoch 3/4

```
973/973 ————— 7s 7ms/step - loss: 0.0167
```

Epoch 4/4

```
973/973 ————— 8s 8ms/step - loss: 0.0139
```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f002b7b7a10>
```

```
In [ ]: # Scale the entire dataset using the same scaler fitted on the training data
scaled_full_data = scaler.transform(df[numerical_cols])

forecast_gru = []
#Replace periods with whatever forecast length you want
periods = 4
first_eval_batch = scaled_full_data[-length:]
```

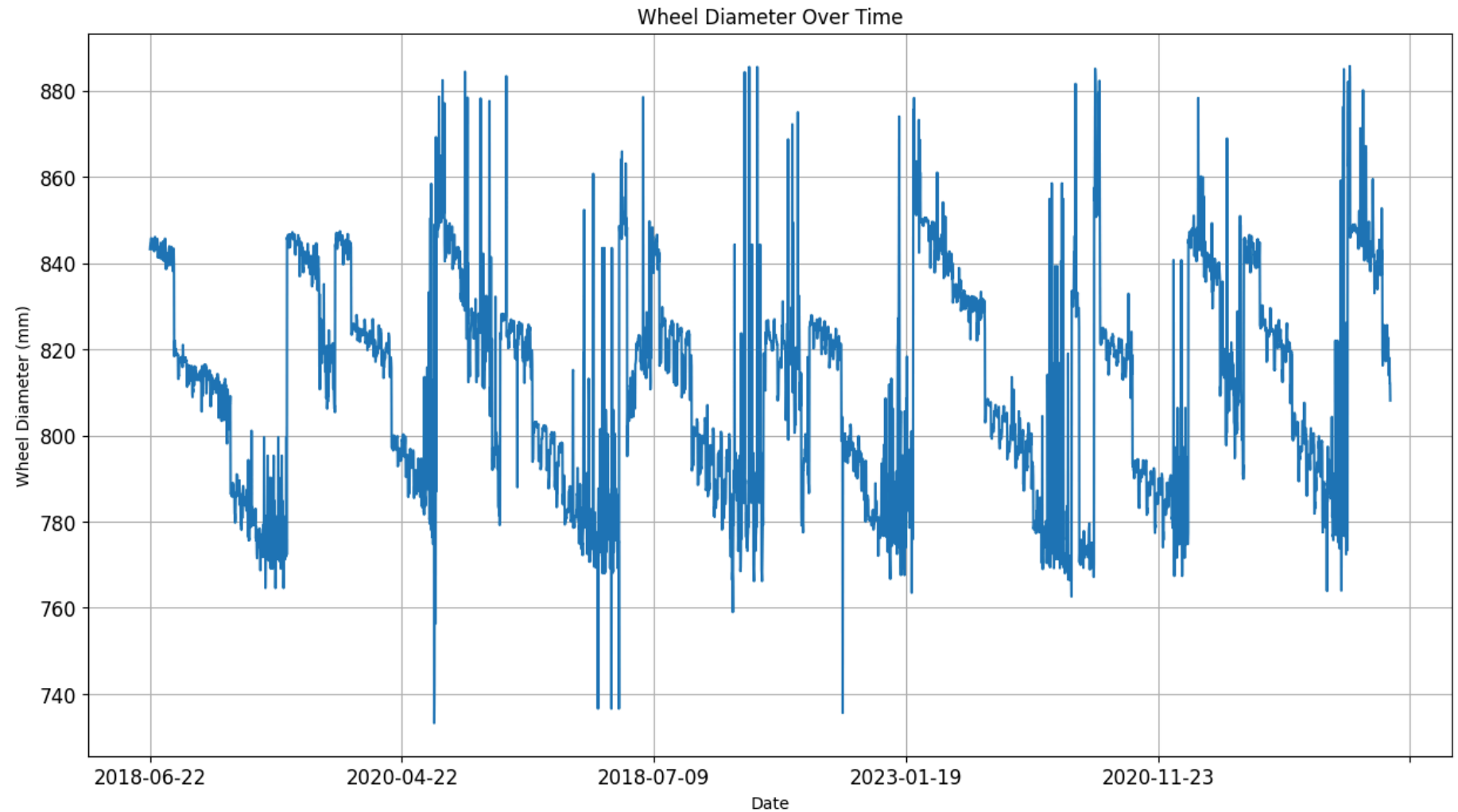
```
current_batch = first_eval_batch.reshape((1, length,n_features))

for i in range(periods):
    #get prediction 1 time stamp ahead
    current_pred_gru = gru_model.predict(current_batch, verbose=0)[0] # Added verbose=0 to suppress output

    forecast_gru.append(current_pred_gru)
    #update batch to now include prediction and drop first value
    current_batch = np.append(current_batch[:,1:,:],[[current_pred_gru]],axis=1) # Changed current_pred to current_pred_gru
```

WHEEL DIAMETER

```
In [ ]: #plot the time series data
df.set_index('Date', inplace=True) # Make sure 'Date' is parsed to datetime
df['WheelDiameter'].plot(figsize=(15, 8), title='Wheel Diameter Over Time', fontsize=12)
plt.ylabel('Wheel Diameter (mm)')
plt.grid(True)
```



```
In [ ]: len(df)
```

```
Out[ ]: 4923
```

TRAIN AND TEST SPLIT

```
In [ ]: test_size = 3938  
test_ind = len(df) - test_size
```

```
In [ ]: train = df.iloc[:test_ind]
        test = df.iloc[test_ind:]
```

```
In [ ]: #Feature scalling
        from sklearn.preprocessing import MinMaxScaler
        numerical_cols = ['WheelDiameter']
        scaler = MinMaxScaler()
        scaler.fit(train[numerical_cols])
```

```
Out[ ]: ▼ MinMaxScaler ⓘ ⓘ
        MinMaxScaler()
```

```
In [ ]: from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator
        length = 12

        # Scale the training data and assign it to scaled_train
        scaled_train = scaler.transform(train[numerical_cols])

        generator = TimeseriesGenerator(scaled_train, scaled_train, length=length, batch_size=1)
```

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, LSTM, GRU
```

```
In [ ]: from keras.layers import Dropout
        from keras.optimizers import Adam

        gru_model = Sequential()

        # Define the number of features
        n_features = 1 # Get the number of columns from the scaled training data

        gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
        gru_model.add(Dropout(0.2))

        gru_model.add(Dense(1))

        optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
```

```
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.summary()
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
gru_2 (GRU)	(None, 50)	7,950
dropout_2 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 1)	51

Total params: 8,001 (31.25 KB)

Trainable params: 8,001 (31.25 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: # Scale the test data using the same scaler fitted on the training data
scaled_test = scaler.transform(test[numerical_cols])

# Create a TimeseriesGenerator for the validation data
# The targets for the validation generator should be the scaled test data itself
validation_generator = TimeseriesGenerator(scaled_test, scaled_test, length=length, batch_size=1)

# Now you can call the fit method with the validation_generator
gru_model.fit(generator, epochs=20, validation_data=validation_generator)
```

Epoch 1/20

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self._warn_if_super_not_called()
```



```

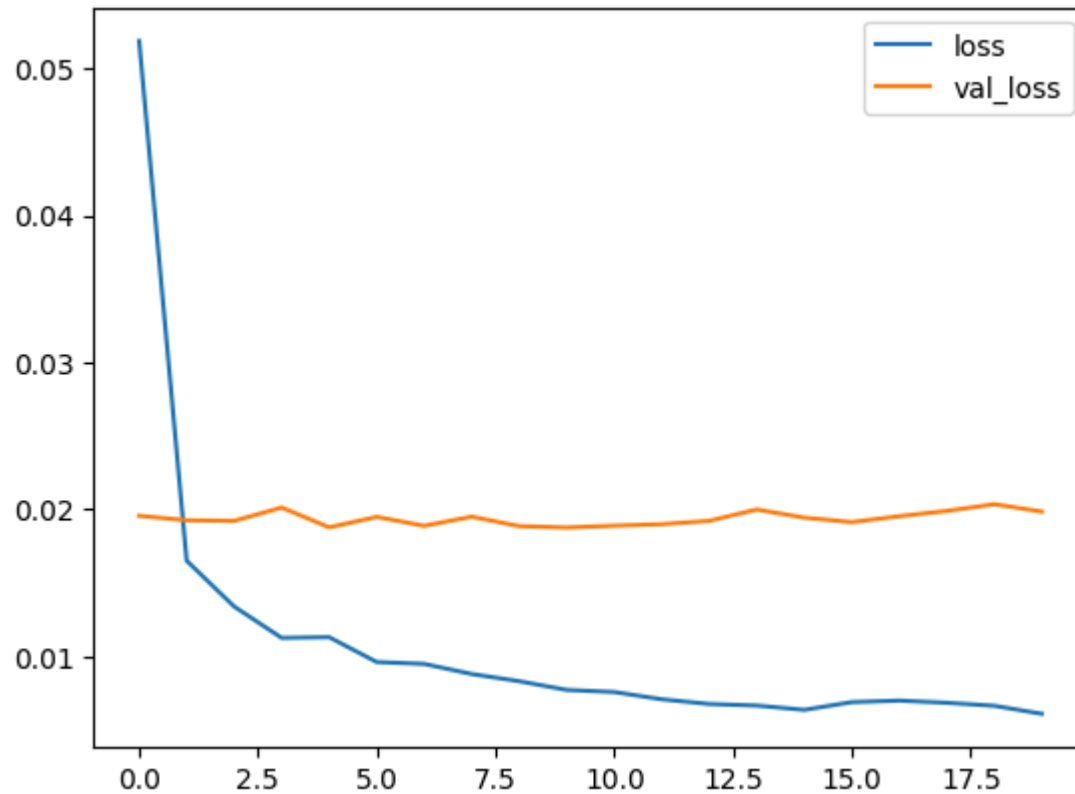
973/973 ————— 21s 19ms/step - loss: 0.1315 - val_loss: 0.0196
Epoch 2/20
973/973 ————— 20s 18ms/step - loss: 0.0163 - val_loss: 0.0193
Epoch 3/20
973/973 ————— 21s 19ms/step - loss: 0.0133 - val_loss: 0.0192
Epoch 4/20
973/973 ————— 17s 17ms/step - loss: 0.0116 - val_loss: 0.0201
Epoch 5/20
973/973 ————— 30s 30ms/step - loss: 0.0104 - val_loss: 0.0188
Epoch 6/20
973/973 ————— 39s 29ms/step - loss: 0.0105 - val_loss: 0.0195
Epoch 7/20
973/973 ————— 30s 18ms/step - loss: 0.0105 - val_loss: 0.0189
Epoch 8/20
973/973 ————— 17s 18ms/step - loss: 0.0096 - val_loss: 0.0195
Epoch 9/20
973/973 ————— 19s 19ms/step - loss: 0.0100 - val_loss: 0.0189
Epoch 10/20
973/973 ————— 27s 28ms/step - loss: 0.0094 - val_loss: 0.0188
Epoch 11/20
973/973 ————— 17s 17ms/step - loss: 0.0080 - val_loss: 0.0189
Epoch 12/20
973/973 ————— 19s 19ms/step - loss: 0.0071 - val_loss: 0.0190
Epoch 13/20
973/973 ————— 17s 18ms/step - loss: 0.0067 - val_loss: 0.0192
Epoch 14/20
973/973 ————— 19s 20ms/step - loss: 0.0069 - val_loss: 0.0200
Epoch 15/20
973/973 ————— 17s 18ms/step - loss: 0.0062 - val_loss: 0.0195
Epoch 16/20
973/973 ————— 18s 18ms/step - loss: 0.0080 - val_loss: 0.0192
Epoch 17/20
973/973 ————— 20s 18ms/step - loss: 0.0062 - val_loss: 0.0196
Epoch 18/20
973/973 ————— 17s 18ms/step - loss: 0.0069 - val_loss: 0.0199
Epoch 19/20
973/973 ————— 18s 18ms/step - loss: 0.0068 - val_loss: 0.0204
Epoch 20/20
973/973 ————— 17s 17ms/step - loss: 0.0057 - val_loss: 0.0199

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f002ae72650>
```

```
In [ ]: losses = pd.DataFrame(gru_model.history.history)
losses.plot()
```

Out[]: <Axes: >



```
In [ ]: test_gru_predictions = []

first_eval_batch = scaled_train[-length:]
current_batch = first_eval_batch.reshape((1, length, n_features))

for i in range(len(test)):
    #get prediction 1 time stamp ahead
    current_pred = gru_model.predict(current_batch, verbose=0)[0] # Changed 'model' to 'gru_model' and added verbose=0 to suppress
    #store prediction
    test_gru_predictions.append(current_pred)
```

```
#update batch to now include prediction and drop first value  
current_batch = np.append(current_batch[:,1:,:],[[current_pred]],axis=1)
```

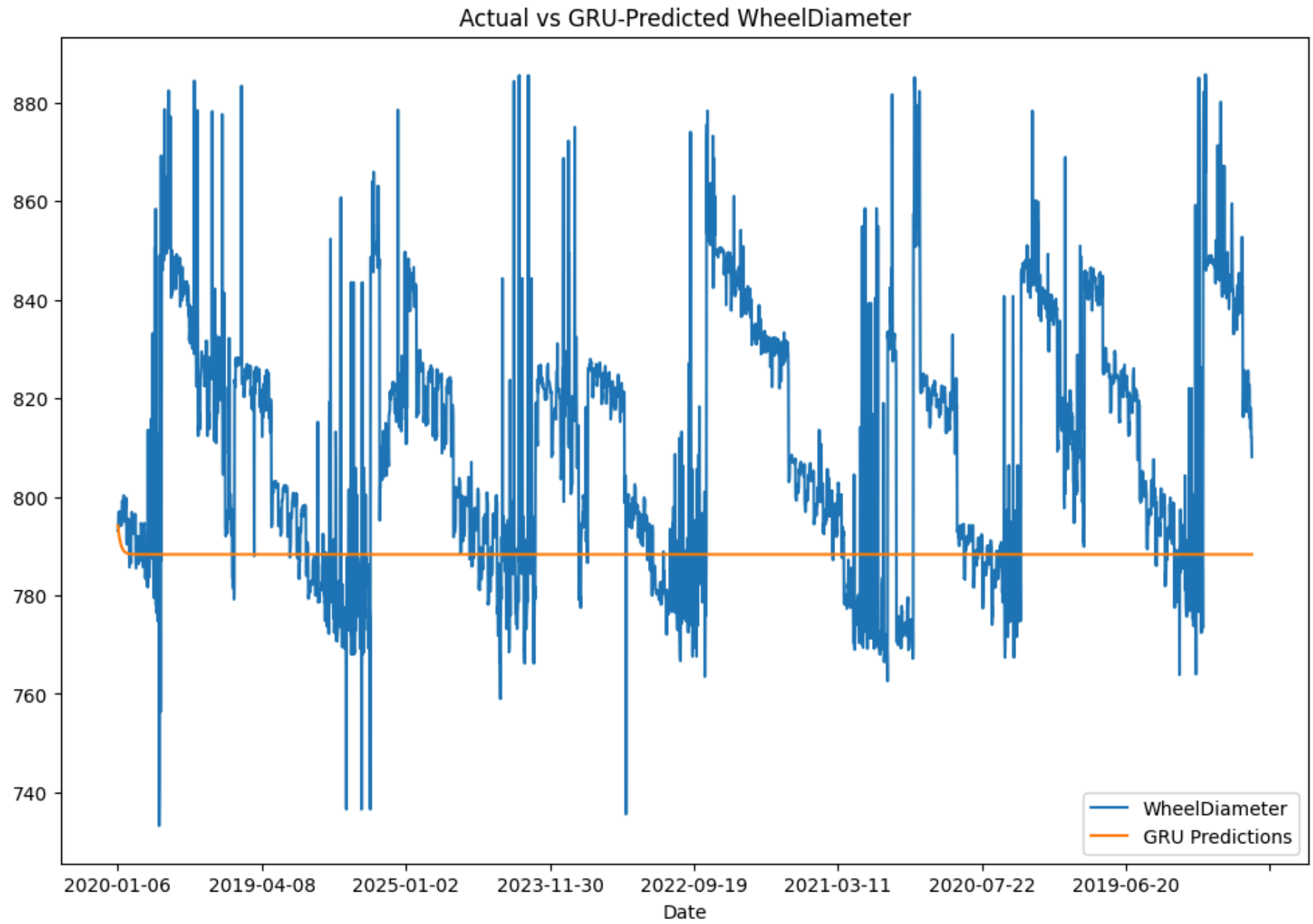
```
In [ ]: true_prediction = scaler.inverse_transform(test_gru_predictions)  
test['GRU Predictions'] = true_prediction  
test[['WheelDiameter', 'GRU Predictions']].plot(figsize=(12,8), title='Actual vs GRU-Predicted WheelDiameter')
```

```
<ipython-input-33-4fc434aa9f3c>:2: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
test['GRU Predictions'] = true_prediction
```

```
Out[ ]: <Axes: title={'center': 'Actual vs GRU-Predicted WheelDiameter'}, xlabel='Date'>
```



```
In [ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from numpy import sqrt
```

```
In [ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from numpy import sqrt
```

```
# Inverse transform predictions if needed
true_prediction = scaler.inverse_transform(test_gru_predictions)
test['GRU Predictions'] = true_prediction.flatten()

# Inverse transform actual values if WheelDiameter is still scaled
# Uncomment the line below if your 'WheelDiameter' column is still scaled
# test['WheelDiameter'] = scaler.inverse_transform(test[['WheelDiameter']])

# Ground truth and prediction arrays
y_true = test['WheelDiameter'].values
y_pred = test['GRU Predictions'].values

# Metrics calculation
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mse = mean_squared_error(y_true, y_pred)
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
r2 = r2_score(y_true, y_pred)

# Print results
print(f"MAE: {mae:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MSE: {mse:.4f}")
print(f"MAPE: {mape:.2f}%")
print(f"R²: {r2:.4f}")
```

```
MAE: 25.9673
RMSE: 32.9390
MSE: 1084.9745
MAPE: 3.14%
R²: -0.7512
```

```
<ipython-input-35-acdd81adeaf5>:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
test['GRU Predictions'] = true_prediction.flatten()
```

```
In [ ]: gru_model = Sequential()
gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
gru_model.add(Dropout(0.2))

gru_model.add(Dense(1))

optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.fit(generator, epochs=4)
```

Epoch 1/4

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(**kwargs)
```

```
973/973 ————— 10s 8ms/step - loss: 0.0726
```

Epoch 2/4

```
973/973 ————— 7s 7ms/step - loss: 0.0127
```

Epoch 3/4

```
973/973 ————— 10s 7ms/step - loss: 0.0088
```

Epoch 4/4

```
973/973 ————— 8s 8ms/step - loss: 0.0094
```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f002ae1ed10>
```

```
In [ ]: # Scale the entire dataset using the same scaler fitted on the training data
scaled_full_data = scaler.transform(df[numerical_cols])

forecast_gru = []
#Replace periods with whatever forecast length you want
periods = 4
first_eval_batch = scaled_full_data[-length:]
```

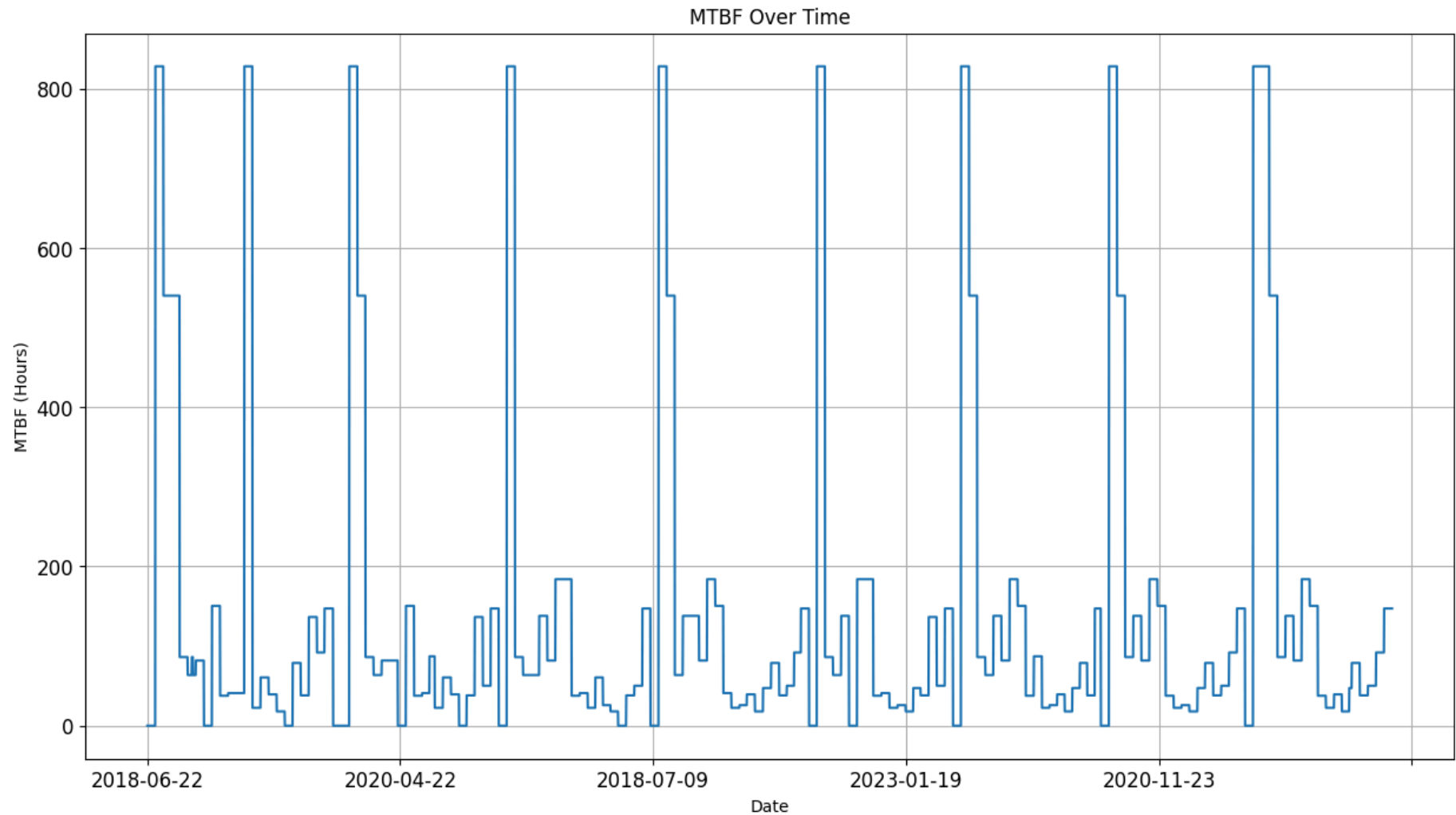
```
current_batch = first_eval_batch.reshape((1, length,n_features))

for i in range(periods):
    #get prediction 1 time stamp ahead
    current_pred_gru = gru_model.predict(current_batch, verbose=0)[0] # Added verbose=0 to suppress output

    forecast_gru.append(current_pred_gru)
    #update batch to now include prediction and drop first value
    current_batch = np.append(current_batch[:,1:,:],[[current_pred_gru]],axis=1) # Changed current_pred to current_pred_gru
```

GRU MTBF

```
In [ ]: #plot the time series data
#df.set_index('Date', inplace=True) # Make sure 'Date' is parsed to datetime
df['MTBF'].plot(figsize=(15, 8), title='MTBF Over Time', fontsize=12)
plt.ylabel('MTBF (Hours)')
plt.grid(True)
```



```
In [ ]: len(df)
```

```
Out[ ]: 4923
```

TRAIN TEST SPLIT

```
In [ ]: test_size = 3938  
test_ind = len(df) - test_size
```



```
In [ ]: train = df.iloc[:test_ind]
        test = df.iloc[test_ind:]
```

```
In [ ]: #Feature scalling
        from sklearn.preprocessing import MinMaxScaler
        numerical_cols = ['MTBF']
        scaler = MinMaxScaler()
        scaler.fit(train[numerical_cols])
```

```
Out[ ]: ▼ MinMaxScaler ⓘ ⓘ
        MinMaxScaler()
```

```
In [ ]: from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator
        length = 12

        # Scale the training data and assign it to scaled_train
        scaled_train = scaler.transform(train[numerical_cols])

        generator = TimeseriesGenerator(scaled_train,scaled_train,length=length,batch_size=1)
```

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense,LSTM,GRU
```

```
In [ ]: from keras.layers import Dropout
        from keras.optimizers import Adam

        gru_model = Sequential()

        # Define the number of features
        n_features = 1 # Get the number of columns from the scaled training data

        gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
        gru_model.add(Dropout(0.2))

        gru_model.add(Dense(1))

        optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
```

```
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.summary()
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
gru_4 (GRU)	(None, 50)	7,950
dropout_4 (Dropout)	(None, 50)	0
dense_4 (Dense)	(None, 1)	51

Total params: 8,001 (31.25 KB)

Trainable params: 8,001 (31.25 KB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: # Scale the test data using the same scaler fitted on the training data
scaled_test = scaler.transform(test[numerical_cols])

# Create a TimeseriesGenerator for the validation data
# The targets for the validation generator should be the scaled test data itself
validation_generator = TimeseriesGenerator(scaled_test, scaled_test, length=length, batch_size=1)

# Now you can call the fit method with the validation_generator
gru_model.fit(generator, epochs=20, validation_data=validation_generator)
```

Epoch 1/20

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self._warn_if_super_not_called()
```

```

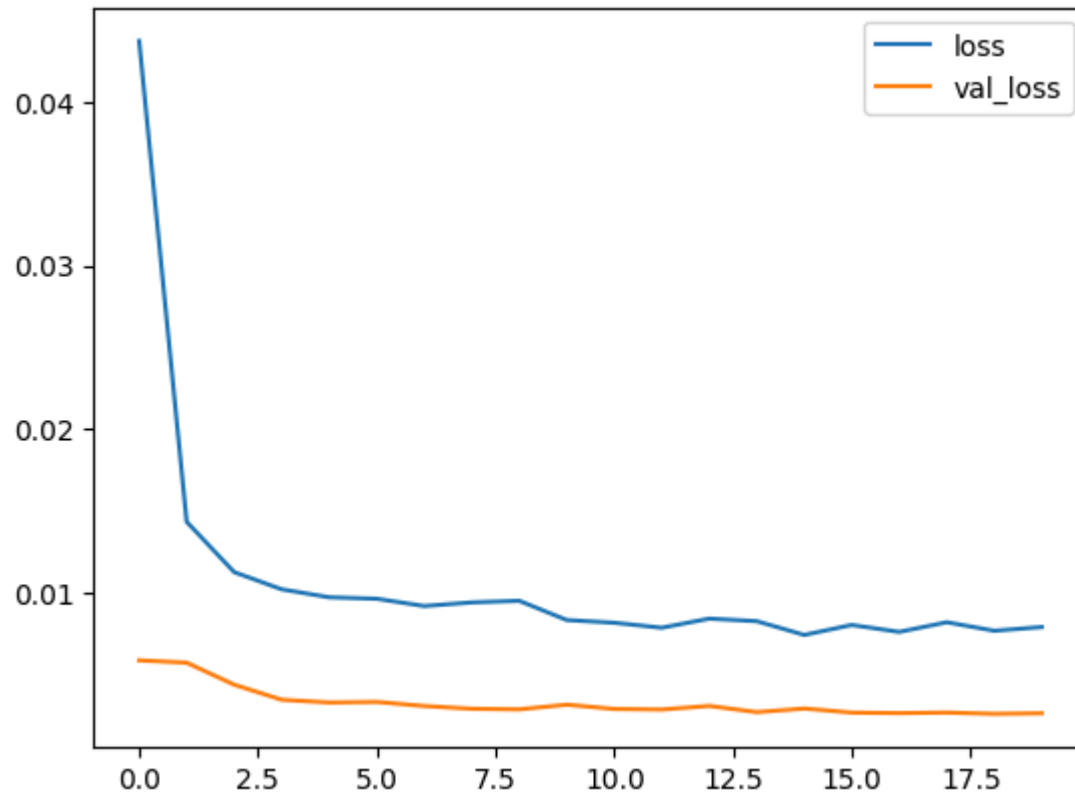
973/973 ————— 21s 18ms/step - loss: 0.0900 - val_loss: 0.0059
Epoch 2/20
973/973 ————— 20s 18ms/step - loss: 0.0174 - val_loss: 0.0057
Epoch 3/20
973/973 ————— 28s 29ms/step - loss: 0.0097 - val_loss: 0.0044
Epoch 4/20
973/973 ————— 18s 18ms/step - loss: 0.0134 - val_loss: 0.0035
Epoch 5/20
973/973 ————— 17s 18ms/step - loss: 0.0089 - val_loss: 0.0033
Epoch 6/20
973/973 ————— 20s 18ms/step - loss: 0.0094 - val_loss: 0.0033
Epoch 7/20
973/973 ————— 18s 18ms/step - loss: 0.0089 - val_loss: 0.0031
Epoch 8/20
973/973 ————— 20s 18ms/step - loss: 0.0105 - val_loss: 0.0029
Epoch 9/20
973/973 ————— 21s 21ms/step - loss: 0.0086 - val_loss: 0.0029
Epoch 10/20
973/973 ————— 18s 19ms/step - loss: 0.0096 - val_loss: 0.0032
Epoch 11/20
973/973 ————— 18s 19ms/step - loss: 0.0073 - val_loss: 0.0029
Epoch 12/20
973/973 ————— 30s 28ms/step - loss: 0.0067 - val_loss: 0.0029
Epoch 13/20
973/973 ————— 18s 18ms/step - loss: 0.0086 - val_loss: 0.0031
Epoch 14/20
973/973 ————— 30s 28ms/step - loss: 0.0099 - val_loss: 0.0027
Epoch 15/20
973/973 ————— 18s 18ms/step - loss: 0.0077 - val_loss: 0.0029
Epoch 16/20
973/973 ————— 19s 19ms/step - loss: 0.0060 - val_loss: 0.0027
Epoch 17/20
973/973 ————— 18s 19ms/step - loss: 0.0081 - val_loss: 0.0026
Epoch 18/20
973/973 ————— 20s 20ms/step - loss: 0.0063 - val_loss: 0.0027
Epoch 19/20
973/973 ————— 18s 18ms/step - loss: 0.0069 - val_loss: 0.0026
Epoch 20/20
973/973 ————— 18s 19ms/step - loss: 0.0099 - val_loss: 0.0026

```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f0029ab4fd0>
```

```
In [ ]: losses = pd.DataFrame(gru_model.history.history)
        losses.plot()
```

Out[]: <Axes: >



```
In [ ]: test_gru_predictions = []

first_eval_batch = scaled_train[-length:]
current_batch = first_eval_batch.reshape((1, length, n_features))

for i in range(len(test)):
    #get prediction 1 time stamp ahead
    current_pred = gru_model.predict(current_batch, verbose=0)[0] # Changed 'model' to 'gru_model' and added verbose=0 to suppress
    #store prediction
    test_gru_predictions.append(current_pred)
```

```
#update batch to now include prediction and drop first value  
current_batch = np.append(current_batch[:,1:,:], [[current_pred]], axis=1)
```

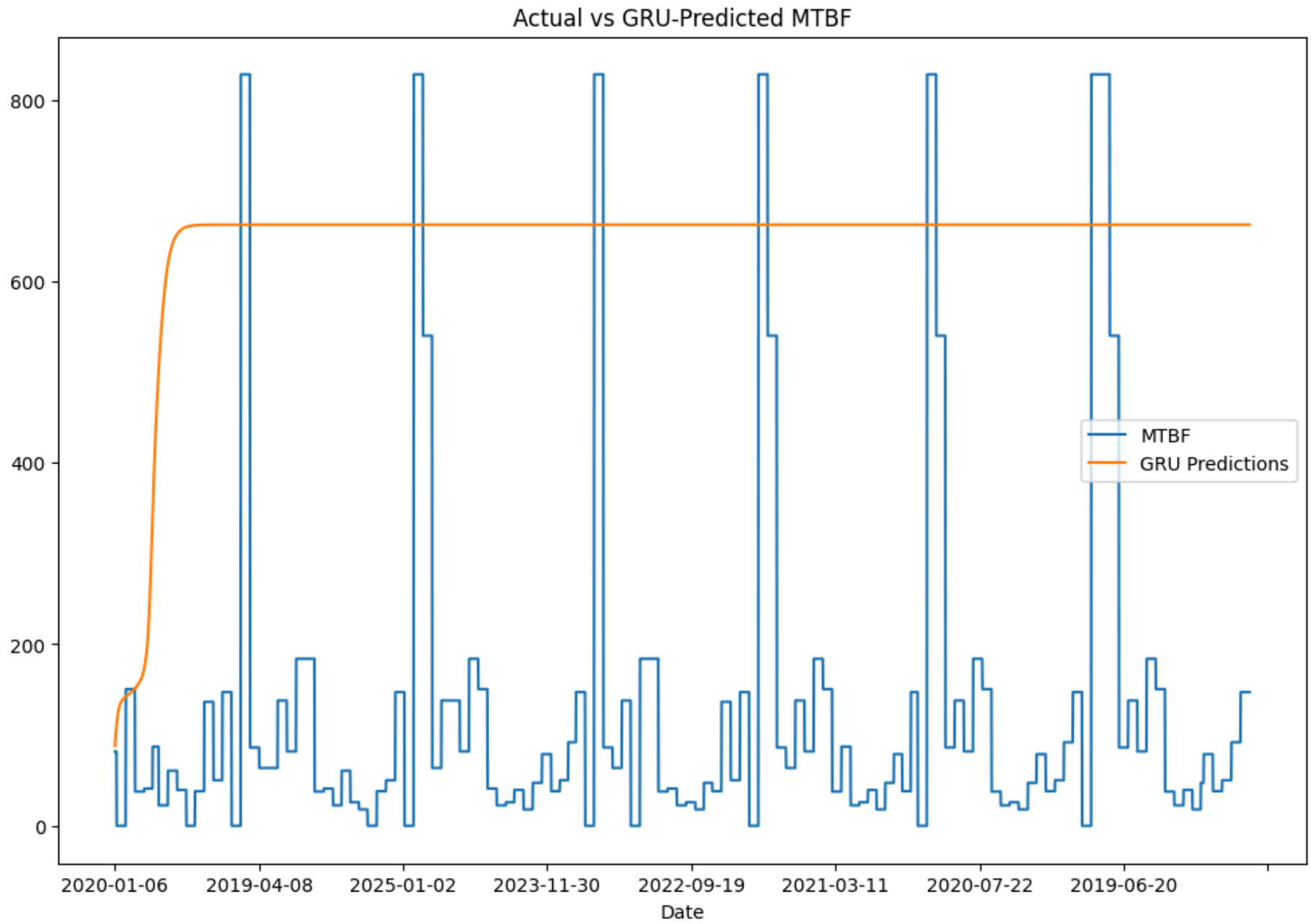
```
In [ ]: true_prediction = scaler.inverse_transform(test_gru_predictions)  
test['GRU Predictions'] = true_prediction  
test[['MTBF', 'GRU Predictions']].plot(figsize=(12,8), title='Actual vs GRU-Predicted MTBF')
```

```
<ipython-input-52-cba65b70fce7>:2: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
test['GRU Predictions'] = true_prediction
```

```
Out[ ]: <Axes: title={'center': 'Actual vs GRU-Predicted MTBF'}, xlabel='Date'>
```



EVALUATION

```
In [ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from numpy import sqrt

# Inverse transform predictions if needed
true_prediction = scaler.inverse_transform(test_gru_predictions)
test['GRU Predictions'] = true_prediction.flatten()

# Inverse transform actual values if MTBF is still scaled
# Uncomment the line below if your 'MTBF' column is still scaled
# test['MTBF'] = scaler.inverse_transform(test[['MTBF']])

# Ground truth and prediction arrays
y_true = test['MTBF'].values
y_pred = test['GRU Predictions'].values

# Metrics calculation
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mse = mean_squared_error(y_true, y_pred)
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
r2 = r2_score(y_true, y_pred)

# Print results
print(f"MAE: {mae:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"MSE: {mse:.4f}")
print(f"MAPE: {mape:.2f}%")
print(f"R²: {r2:.4f}")
```

```
MAE: 531.7551
RMSE: 554.6376
MSE: 307622.8401
MAPE: inf%
R²: -6.9064
```

```
<ipython-input-53-23c18ba68f22>:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
test['GRU Predictions'] = true_prediction.flatten()
<ipython-input-53-23c18ba68f22>:21: RuntimeWarning: divide by zero encountered in divide
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
In [ ]: gru_model = Sequential()
gru_model.add(GRU(50, activation='relu', input_shape=(length, n_features)))
gru_model.add(Dropout(0.2))

gru_model.add(Dense(1))

optimizer = Adam(learning_rate=0.0005) #default is 0.001;try lowering it to 0.0005
gru_model.compile(optimizer=optimizer, loss='mse')

gru_model.fit(generator, epochs=4)
```

Epoch 1/4

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(**kwargs)
973/973 ————— 10s 8ms/step - loss: 0.0750
Epoch 2/4
973/973 ————— 8s 8ms/step - loss: 0.0112
Epoch 3/4
973/973 ————— 7s 7ms/step - loss: 0.0108
Epoch 4/4
973/973 ————— 8s 8ms/step - loss: 0.0112
```

```
Out[ ]: <keras.src.callbacks.history.History at 0x7f002b4a7b10>
```

```
In [ ]: # Scale the entire dataset using the same scaler fitted on the training data
scaled_full_data = scaler.transform(df[numerical_cols])

forecast_gru = []
#Replace periods with whatever forecast length you want
```



```
periods = 4
first_eval_batch = scaled_full_data[-length:]
current_batch = first_eval_batch.reshape((1, length, n_features))

for i in range(periods):
    #get prediction 1 time stamp ahead
    current_pred_gru = gru_model.predict(current_batch, verbose=0)[0] # Added verbose=0 to suppress output

    forecast_gru.append(current_pred_gru)
    #update batch to now include prediction and drop first value
    current_batch = np.append(current_batch[:,1:,:], [[current_pred_gru]], axis=1) # Changed current_pred to current_pred_gru
```